1 Introduction

The MIET-GUI (*metal-induced energy transfer graphical user interface*) is a software packet written in the MATLAB programming language. It is intended as a user-friendly way to convert lifetime-images of arbitrary dipole emitters into height-images. The theory behind this software packet is described in (Buchkapitel). Briefly, a dipole emitter can transfer energy to nearby metal surfaces, thus changing its excited state lifetime. This near-field effect is distance dependant with an almost linear regime in the first tens of nanometers, depending on the wavelength of the emitter and the properties of the sample. Our software uses the relationship between lifetime and height of the emitter above the surface to transform lifetime information into height information.
There are two basic evaluation modes available, pixel-by-pixel mode and single-molecule mode. In the pixel-by-pixel mode, it is assumed that there are many, freely rotating emitters. The height information gained for a single pixel is then determined by all emitters whose fluorescence signal contributes to this pixel. One example is a sample consisting of a cell membrane that has been stained using antibodies. The single-molecule mode assumes that there is only a small number of emitters in the sample and that the fluorescence of each emitter covers several pixels on the camera image. In this case, the height of each fluorescent molecule is determined by combining the signal from all pixels belonging to this emitter. One example of this is a strongly diluted dye solution spin-coated onto a coverslide.
In the following sections, we describe how to use the MIET-GUI in either evaluation mode.

2 Pixel-by-pixel mode

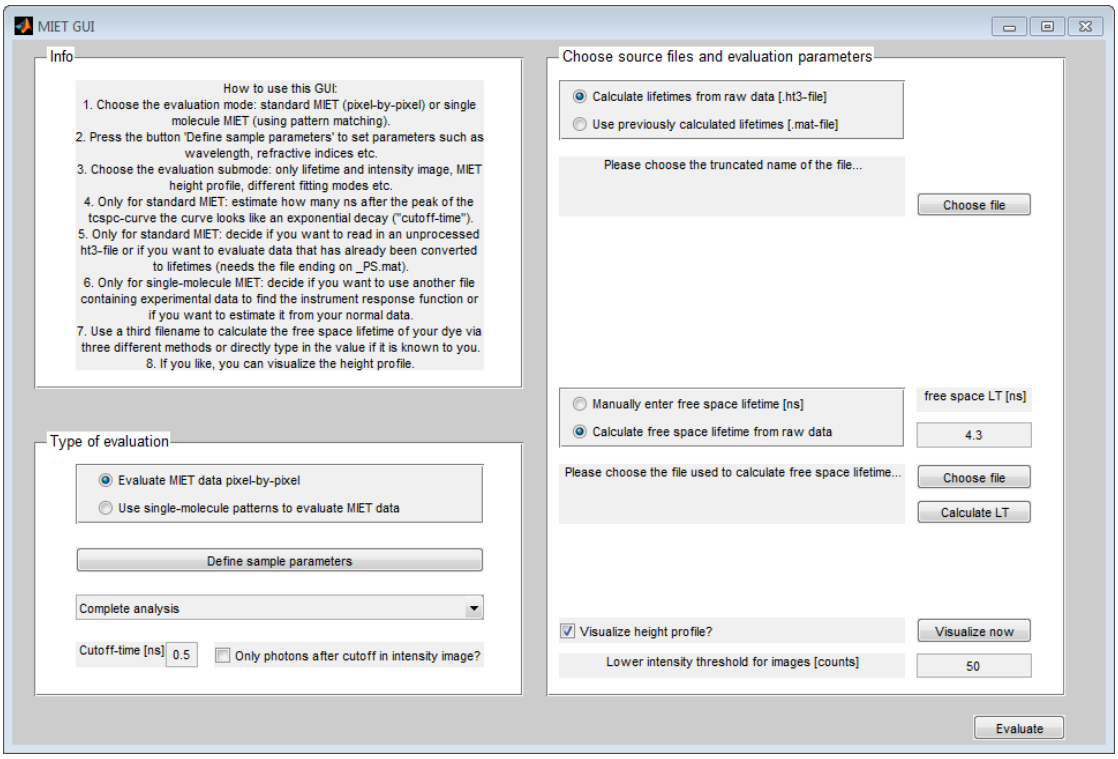


Figure 1 – Graphical user interface as seen in the pixel-by-pixel mode.

Open MATLAB and run the file `MIET_gui.m`. A graphical interface as in figure 1 will open. This interface has three main areas:

Info: A short help explaining how to use the GUI.

Type of evaluation: At the top, there are radio buttons to choose between the pixel-by-pixel mode and the single-molecule mode. Depending on this choice, the layout of the GUI will change slightly. Below, there is a button labeled `Define sample parameters`. Its use is described in section 4. Next, there is a drop-down menu that offers a range of sub-evaluation modes: complete analysis, only generation of lifetime and intensity images from the raw ht3-file, only calculation of the MIET calibration curve. The last one is especially useful when planning an experiment because it can be used to show the influence of sample parameters (type of metal, thickness of metal film etc.) on the shape of the MIET calibration curve and thereby on the sensitivity of the method to certain height ranges.
Finally, the cutoff-time can be changed. This parameter influences the calculation of lifetimes from time-correlated single-photon counting (TCSPC) histograms. The TCSPC curve of a single pixel can be described by a steep rise, followed by a peak and then an exponential decay (see figure 2). By setting a cutoff-time after the peak after which the curve is purely exponential and fitting the TCSPC curve with an exponential function[1] one gets the lifetime value for this pixel. During the evaluation, there is a plot produced for each detector that shows the TCSPC curve of this detector together with the cutoff, enabling the user to check if the cutoff was chosen well.
When ticking the checkbox `Only photons after cutoff in intensity image?`,

[1] More precisely, we do a multiexponential fit: a linear least-squares fit with a nonnegativity contraint determines the relative contributions of different lifetimes, the weighted average of these lifetimes is the value that is assigned to the pixel.

the intensity images produced by the GUI will only contain photons that arrived after the cutoff-time. This can strongly reduce the background in samples with a high amount of scattering (e.g. gold-coated cover slides).

Choose source files and evaluation parameters: The first set of radio buttons offers a choice between using a raw ht3-file as input for the analysis or using a previously calculated lifetime and intensity image saved in a mat-file. The latter has to contain two variables, `tag` and `life_imm`. They are the intensity and lifetime image, respectively, where the first and second dimension represent the x- and y-direction of the image and the third dimension allows for including several detectors.

The second set of radio buttons concerns the free space lifetime of the emitters[2]. If the free space lifetime is known, it can be directly written into the text field. Otherwise, an ht3-file containing a measurement of the same type of emitters in the same immersion medium but without metal coated cover slides can be used to estimate the free space lifetime. Three methods are used: Firstly, the TCSPC histogram of the whole image is fitted with a multiexponential decay (`tau_fit`). Secondly, the average arrival time of the photons after the cutoff is calculated from the same histogram (`tau_avg`). Thirdly, the lifetime of each single pixel is determined as explained above, these lifetimes are histogrammed and the maximum and mean of this histogram are given as `tau_px_max` and `tau_px_mean`. The user then has to decide which of these results seems most trustworthy and write it manually into the text field.

The last item in this area is the checkbox `Visualize height profile?`. It refers to a pseudo-3d visualization of the height information as in figure 7(d). If this box is ticked, a 3d visualization will happen at the end of the next evaluation of a data set. While the box remains ticked, a text field and a button are visible. The text field contains an intensity threshold, pixels with a lower intensity will not appear in the visualization[3]. The button `Visualize now` immediately produces a 3d image if there are already height and intensity values stored in MATLAB's workspace, that is if a data set has just been evaluated.

Finally, there is a button labelled `Evaluate` at the lower right corner of the GUI. Clicking this button starts the data evaluation.

[2]This quantity is only important for height calculations, so if the evaluation submode is set to `Only show lifetime and intensity images`, it is ignored.

[3]Pixels with low intensities have lifetimes determined by noise, not by an emitter. These lifetimes are therefore random and would produce spikes in the 3d visualization that obscure the actual features of the sample.

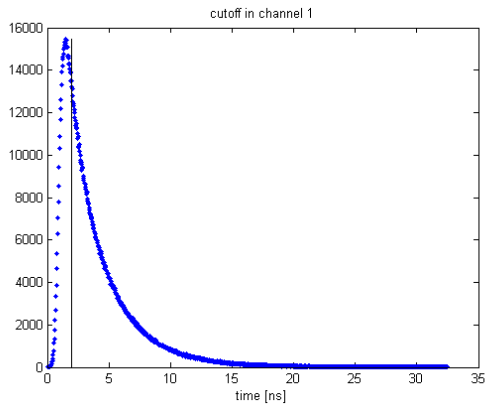


Figure 2 – Combined TCSPC curve of all pixels in a data set. The black line shows the cutoff: Only data points after this time are used for the calculation of the average arrival time of the photons. It is important that the TCSPC curve is purely exponential after this time. The user can shift the cutoff to the right if the peak is very broad to make sure only the exponential part of the curve is used, or to the left if the peak is very narrow and more photons should be used in the evaluation. In this example, the cutoff is 0.5 ns (counted after the peak).

3 Single-molecule mode

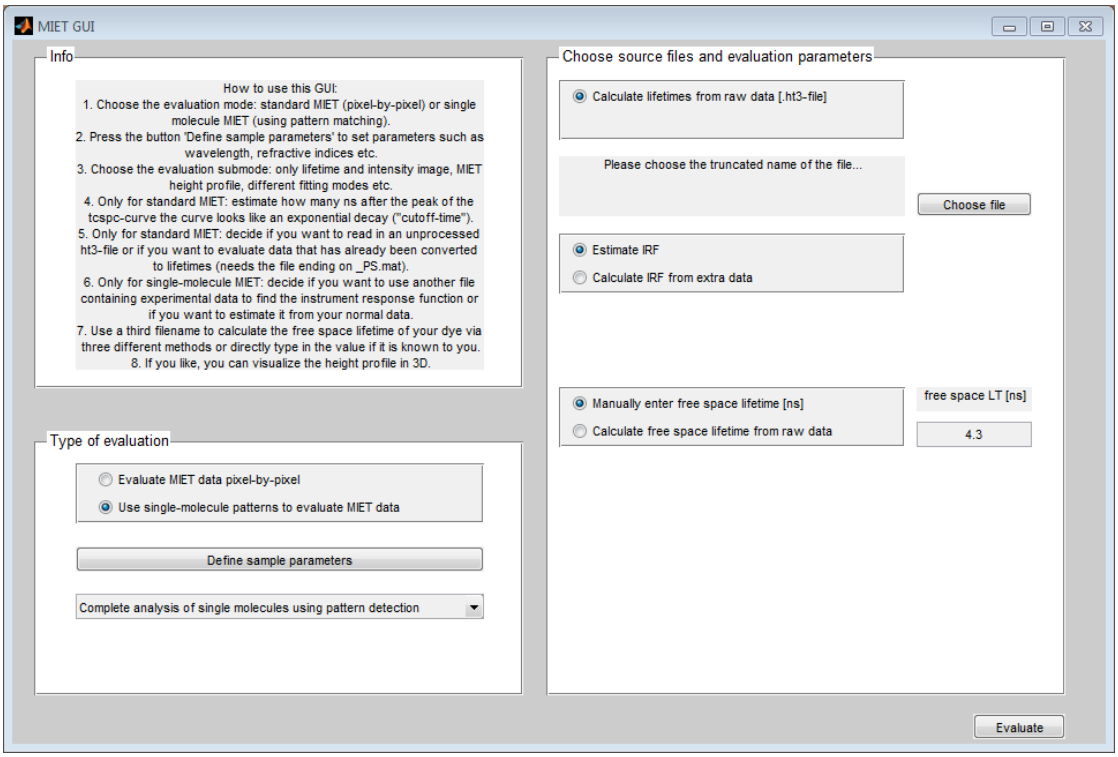


Figure 3 – Graphical user interface as seen in the single-molecule mode.

Open MATLAB and run the file `MIET-gui.m`. A graphical interface as in figure 3 will open. This interface has three main areas:

Info: A short help explaining how to use the GUI.

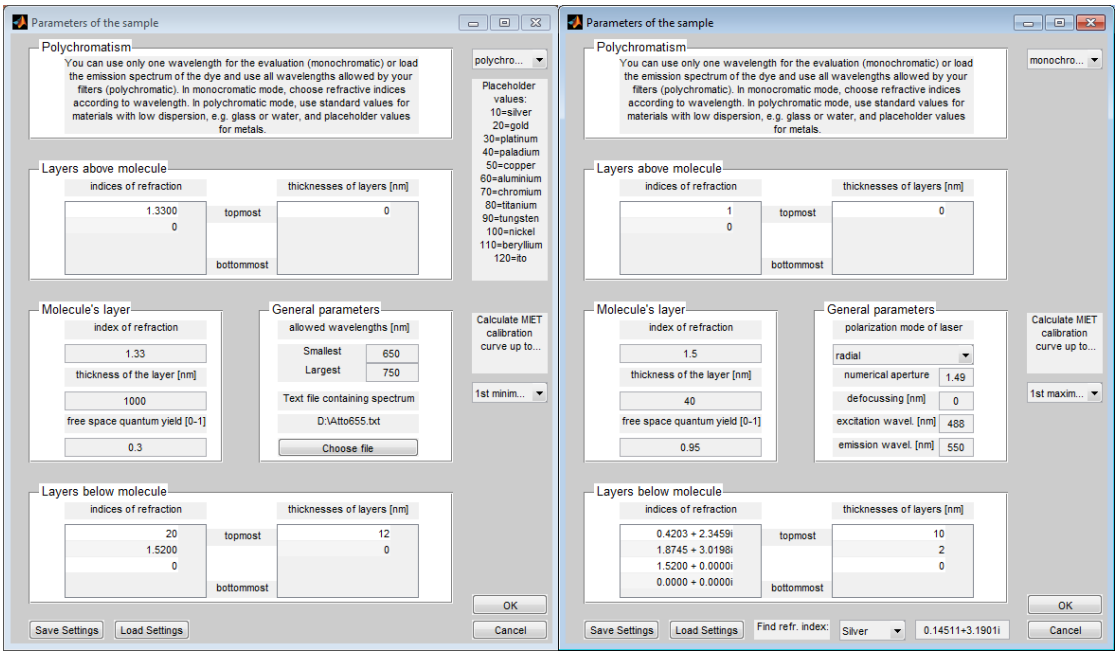
Type of evaluation: At the top, there are radio buttons to choose between the pixel-by-pixel mode and the single-molecule mode. Depending on this choice, the layout of the GUI will change slightly. Below, there is a button labeled '`Define sample parameters`'. Its use is described in section 4. Next, there is a drop-down menu that offers a range of sub-evaluation modes:

- `Complete analysis of single molecules using pattern detection`: Read in a raw ht3-file, find single-molecule intensity patterns, calculate the lifetimes of these molecules, convert them to height information. Display all three quantities.
- `Complete analysis of elliptical regions of interest`: Read in a raw ht3-file, let the user choose elliptical regions of interest in the intensity image, calculate the lifetimes of each of these regions, convert them to height information. Display all three quantities.
- `Only lifetime and intensity using pattern detection, no height`: Read in a raw ht3-file, find single-molecule intensity patterns, calculate the lifetimes of these molecules. Display both quantities.
- `Lifetime and intensity of randomly oriented molecules, e.g. fluorescent beads`: Read in a raw ht3-file, find the intensity patterns of randomly oriented molecules, calculate the lifetimes of the molecules. Display both quantities.
- `Lifetime and intensity of elliptical regions of interest`: Read in a raw ht3-file, let the user choose elliptical regions of interest in the intensity image, calculate the lifetimes of each of these regions. Display both quantities.
- `Find and display patterns and raw intensity image`: Read in a raw ht3-file, find single-molecule intensity patterns, display them.
- `Only show raw intensity image`: Read in a raw ht3-file, display the raw intensity image. Useful for finding good data sets that are worth evaluating.
- `Only show MIET lifetime vs. height calibration curves`: No ht3-file needed, only the sample parameters have to be set. Calculate and display the MIET lifetime vs. height calibration curve for different orientations of the molecule. Useful when planning an experiment because it can be used to show the influence of sample parameters (type of metal, thickness of metal film etc.) on the shape of the MIET calibration curve and thereby on the sensitivity of the method to certain height ranges.

Choose source files and evaluation parameters: Since the most important feature of the single-molecule mode is using single-molecule patterns to improve lifetime calculations, it does not make sense to use pre-calculated lifetimes here. Therefore, the first set of radio buttons is permanently set to `Calculate lifetimes from raw data [.ht3-file]`.
When fitting lifetime curves, it is important to know the instrument response function (IRF). By default, the IRF is estimated from the raw data itself but it is also possible to use an extra measurement. In that case, an ht3-file containing experimental data of a scattering sample (scan or point measurement) is used to calculate a TCSPC curve which is used as the IRF.
The free space lifetime can be taken from literature values / previous measurements or it can be estimated using a further ht3-file. This is explained in section 2.

Finally, there is a button labelled `Evaluate` at the lower right corner of the GUI. Clicking this button starts the data evaluation.

4 Define sample parameters

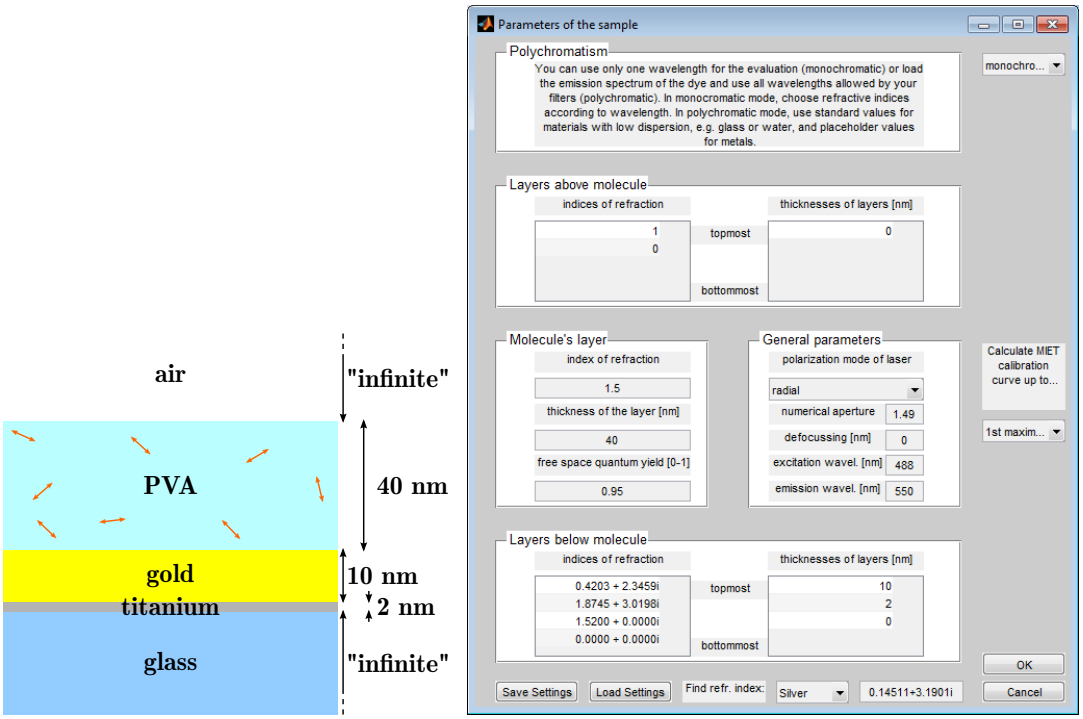


(a) Pixel-by-pixel mode. **(b)** Single-molecule mode.

Figure 4 – Graphical user interface for setting the sample parameters. The only difference between the two evaluation modes can be found in the `General parameters`, where the single-molecule-mode additionaly requires the polarization mode of the laser, the numerical aperture of the objective, the defocussing and the wavelength of the excitation light. The left example shows a polychromatic evaluation of Atto 655 molecules in water above a glass cover slide coated with 12 nm of gold. The right example shows a monochromatic evaluation of Rhodamine 6G molecules in polyvinyl alcohol above a titanium- and gold-coated glass cover slide, see figure 5.

The theoretical calculations behind both pattern matching and the MIET lifetime vs. height calibration curve need some sample properties as input. These properties can be set by clicking the button `Define sample parameters`. A second window opens, either as in figure 4(a) or as in figure 4(b), depending on the evaluation mode.
The first choice is whether the emission of the molecules should be taken as monochromatic or polychromatic. This can be done by the drop-down menu in the top right corner. In the former case, the emission wavelength of the molecules can be entered in a text field under `General parameters`. In the latter case, it is necessary to give a file containing the emission spectrum[4], as well as the lowest and highest wavelength that are to be taken into account (determined, for example, by the filters in the setup).
The second choice is the range of the MIET calibration curve. When the distance between molecule and metal surface increases, the lifetime rises monotonously at first and then starts oscillating with decreasing amplitude about the free space lifetime. As a result, some lifetime values cannot be matched unambiguously to a height value. The first possibility for dealing with this problem is to crop all lifetime values above the lowest minimum of the oscillations and to mark all pixels/molecules with a higher lifetime as "not a number" in the height image (`1st minimum`). If some prior knowledge implies that no fluorescent molecule can be situated higher than the value corresponding to the first maximum of the calibration curve, the curve can be cropped at this value (`1st maximum`). In both cases, the calibration curve is calculated in 1 nm steps. In the pixel-by-pixel mode, it is also possible to choose `manual` and to set the startpoint, endpoint

[4] As an Nx2-matrix, where the first column contains the wavelength in nanometers and the second column the intensity at this wavelength. The intensity does not have to be normalized.



(a) Schematic of the sample. **(b)** Snapshot of the GUI.

Figure 5 – Setting the sample parameters for Rhodamine 6G molecules in PVA above a titanium- and gold-coated glass cover slide. The excitation wavelength is 488 nm, the emission maximum of the dye is at 550 nm and the quantum yield is 95%. The refractive indices at 550 nm are $n_{\mathrm{air}} = 1$, $n_{\mathrm{PVA}} = 1.5$, $n_{\mathrm{Au}} = 0.42 + 2.35\mathrm{i}$, $n_{\mathrm{Ti}} = 1.87 + 3.02\mathrm{i}$, $n_{\mathrm{glass}} = 1.52$ and the thicknesses of the different layers are shown in (a). When fitting emission patterns, the polarization mode of the laser (linear polarization, radially polarized doughnut mode, azimuthally polarized doughnut mode), the numerical aperture of the objective (NA) and the distance the objective is moved out of focus towards the sample (defocussing) are also needed. At the bottom of the window, the refractive index of silver at 550 nm is displayed and can be copied and pasted to the refractive index table. When the emission wavelength is changed, the refractive index is shown at that new wavelength.

and number of steps manually. In this case, if the endpoint is already in the oscillating regime, the cropping of the curve is as in the `1st minimum` mode.
The three sections labelled `Layers above molecule`, `Molecule's layer` and `Layers below molecule` are used to describe the composition of the sample. For each planar layer, such as the glass cover slide, any metal coatings, the buffer solution or polymer layer containing the fluorescent molecules and the air or cover slide above the fluorescent layer, the thickness and refractive index are needed. As an example, see figure 5: On top of a glass cover slide, there is a 2 nm thick layer of titanium, followed by 10 nm of gold. On top of these metal layers, there is a 40 nm layer of polyvinyl alcohol (PVA) containing the dye Rhodamine 6G. Above the polymer layer, there is simply air. In the GUI, the refractive indices and thicknesses of the stacks above and below the dye-containing layer are entered in tables, with the topmost entry corresponding to the topmost material (compare the schematic of the sample to the GUI). The bottommost entry of the layers below the sample (usually the glass cover slide) and the topmost entry of the layers above the sample (usually air or another glass cover slide) are assumed to be much thicker than all other layers. Therefore, their thickness is set to infinity in the caluclations and no thickness value may be entered in the GUI. This is why the refractive index tables always contain one more entry than the thickness tables. If there is no layer above the molecule's layer, for example because the molecules have been spin-coated onto a cover slide and are now surrounded by air, the same refractive index has

to be given for `Molecule's layer` and `Layers above molecule`. Also, the thickness value entered in `Molecule's layer` does not matter in this case (see figure 4(a), where 1000 nm was given as an arbitrary thickness value). The last entry in all four tables is always a zero. This has technical reasons, it allows the user to click on this number to enter new values for new layers. In the evaluation, these zeros are simply ignored. Since dispersion is especially strong in metals, the GUI offers refractive indices for a number of metals at wavelengths between 200 and 800 nm: a drop-down menu at the bottom of the window contains the metals, the adjacent text field displays the refractive index at the current emission wavelength. This value can then be copied and pasted to any of the refractive index tables. In polychromatic mode, the refractive index is different at different wavelengths, so entering a single value for any metal would be meaningless. Instead, there are placeholder values telling the program which metal is placed at which position of the stack of materials, allowing it to choose the correct refractive indices for each wavelength.

The section `Molecule's layer` also contains a field for the free space quantum yield of the fluorescent sample, for example 0.3 for Atto 655 or 0.95 for Rhodamine 6G.

Finally, in the pattern matching mode, the section `General parameters` also asks for the polarization mode of the laser (linear polarization, radially polarized doughnut mode, azimuthally polarized doughnut mode), the numerical aperture of the objective (NA), the distance the objective is moved out of focus towards the sample (defocussing) and the excitation wavelength. These parameters are needed to calculate the patterns which are generated when the excitation beam is scanned over single molecules with different orientations.

If the same type of sample has been used for several measurements, it is convenient to save the settings and to simply load them again for the next evaluation. This can be done via the buttons `Save Settings` and `Load Settings`.

The window is closed and the values are used for the evaluation if the user clicks `OK`. If the window is closed by the `Cancel` button or by the normal cross in the top right corner, all changes are lost and the values remain the same as they were before the window was openend.

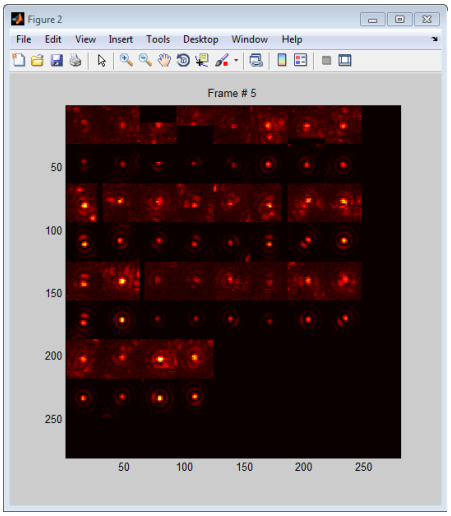
5 Examples

There are a number of example files included in the MIET software packet. They consist of the raw data (`.ht3`-files) and a README containing the sample parameters.

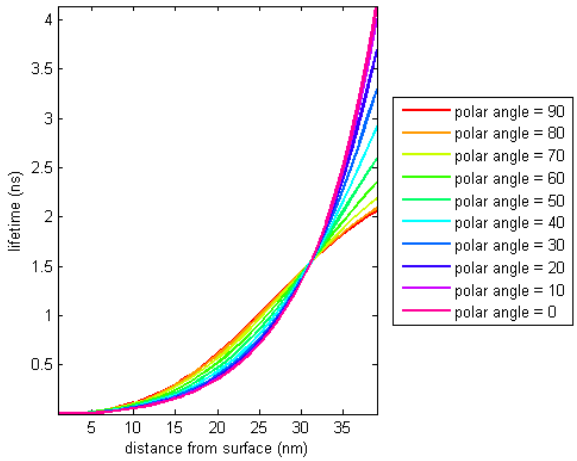
The first example is the same as in figure 5, Rhodamine 6G molecules in polyvinyl alcohol on a titanium- and gold-coated glass cover slide, imaged with a radially polarized doughnut mode of a 488 nm laser. The free space lifetime of Rhodamine 6G in PVA is approximately 3.6 ns. The sample was evaluated in the single-molecule mode, using the evaluation sub-mode `Complete analysis of single molecules using pattern detection`. In the course of this evaluation, the program asks the user to compare eleven possible fit results that have been calculated using slightly different pixels sizes (see figure 6a). The pixel size is taken from the header of the ht3-file, so the approximate value is known, but slight deviations may already strongly influence the fit, so a variation of 0, ± 1, ..., ± 5 nm is shown in the eleven frames. After the best fitted patterns have been chosen by the user, the program finishes the calculations by itself (this may take a few minutes). Afterwards, several windows open, showing the raw intensity image, the fitted patterns, the lifetimes of the molecules, their height inside the polymer layer, a graph of the number of photons per molecule *versus* the lifetimes of the molecules and a graph of the lifetimes *versus* the inclination of the molecules (see figure 8). If the MIET evaluation curves are also of interest, they can be quickly visualized by choosing the evaluation sub-mode `Only show MIET lifetime vs. height calibration curves` and clicking `Evaluate` without changing anything else. The result is shown in figure 6b.

The second example is very simple, Atto 655 molecules on a glass cover slide (free space lifetime 3.6 ns). The left column of figure 9 shows the pixel-by-pixel evaluation of this data set using the evaluation-submode `Complete analysis` with a cutoff of 0.5 ns. The main results are saved in two files ending on `PS.mat` and `height.mat`: The first file contains the intenstiy image `tag` and the lifetime image `life_imm`, the second file contains the height image `height`. The evaluation program only takes into account pixels with at least 26 photons, which is why there are many black pixels in the image showing the "used photons" (figure 9a). The more photons are used for a lifetime estimation, the more accurate the result will be. One can see that the lifetime values in different pixels belonging to the same molecule differ, which might be caused by the fact that not enough photons are used in each pixel. Therefore, it can make sense to combine the photons from all pixels belonging to the same molecule using regions of interest, as shown in the right column of figure 9.

The third example shows mammary epithelial NMuMGS cells treated with transforming growth factor-β (TGFβ) for 48 h. They were stained with the CellMask Deep Red plasma membrane strain (Invitrogen), imaged and evaluated in the polychromatic pixel-by-pixel mode. The results are shown in figure 7: In places with very low count rates, i.e. in the background, the lifetimes fluctuate strongly and may even be negative. In regions with high countrates, the lifetimes are much more homogeneous and result in a convincing height image.

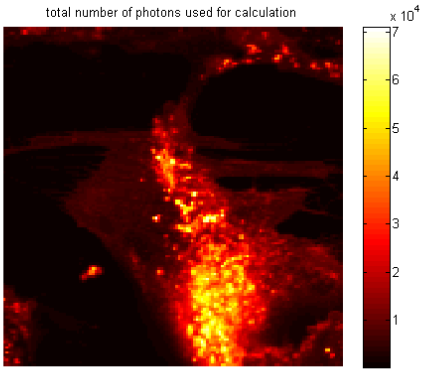


(a) Choosing the best fitted patterns.

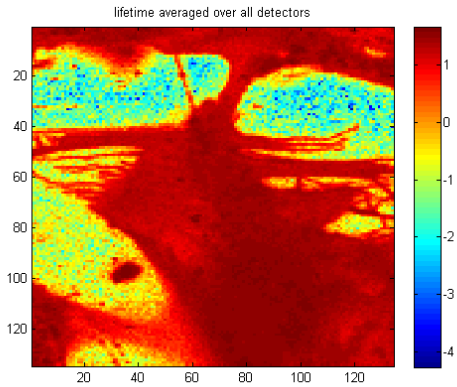


(b) MIET calibration curves for different inclinations.

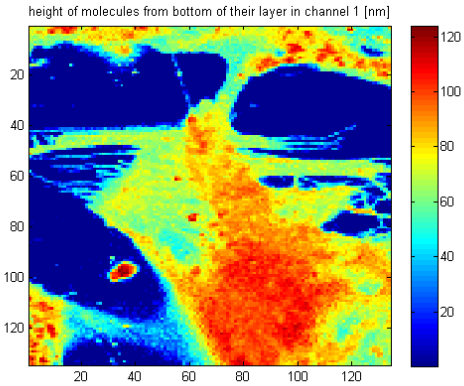
Figure 6 – Required user input when fitting single molecule patterns (a) and MIET calibration curves for different orientations of the fluorescent molecules (b). Both images for the same sample as in figures 5 and 8.



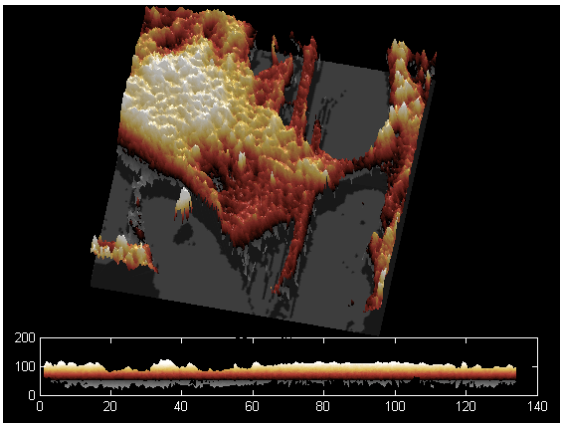
(a) Photons in pixels with >25 photons.



(b) Lifetimes of single pixels (in ns).

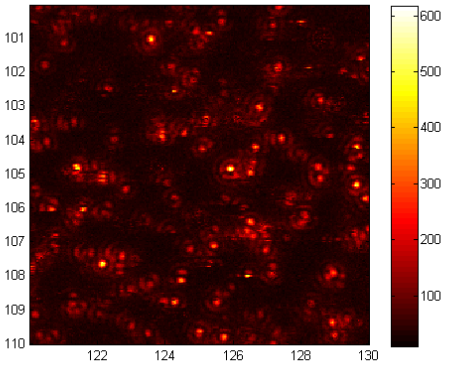


(c) Height values of single pixels (in nm).

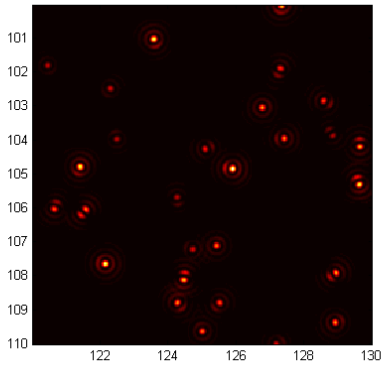


(d) 3d-visualization of the height values.

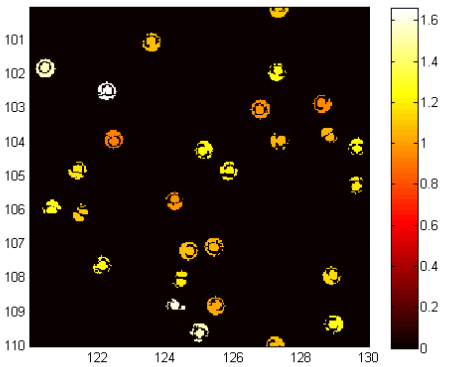
Figure 7 – Mammary epithelial NMuMGS cells treated with TGFβ for 48 h, stained with CellMask Deep Red, imaged and evaluated using the pixel-by-pixel mode.



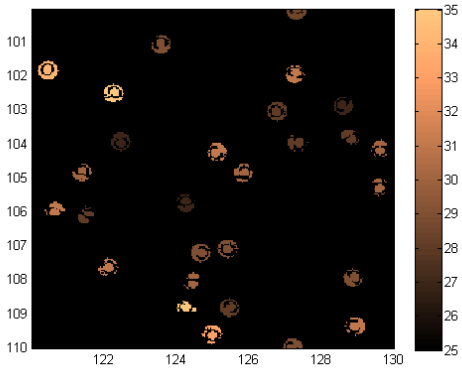
(a) Raw intensity image.



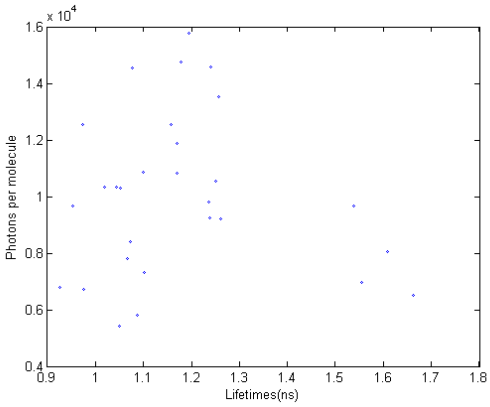
(b) Fitted patterns.



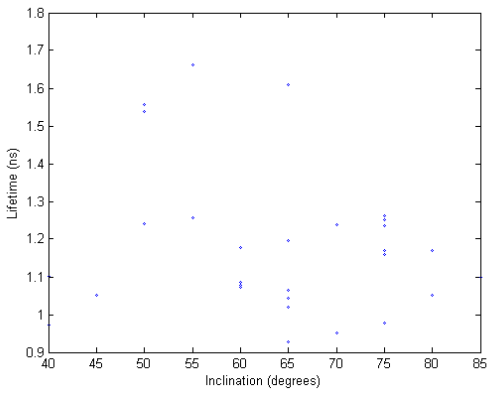
(c) Lifetime image (in ns).



(d) Height image (in nm).

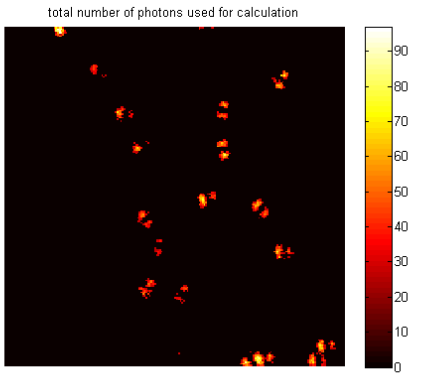


(e) Photon count vs. lifetime.

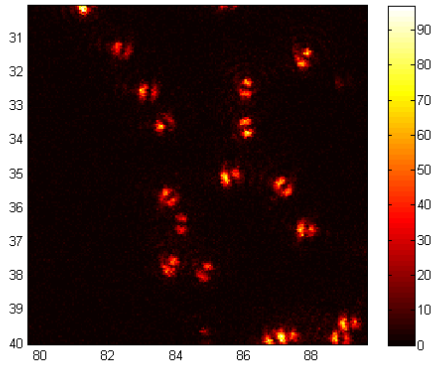


(f) Lifetime vs. inclination.

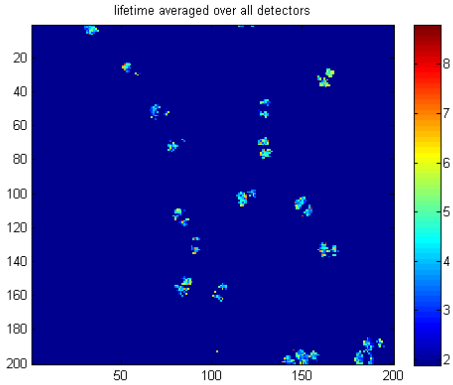
Figure 8 – Output windows produced by the `Complete analysis of single molecules using pattern detection` sub-evaluation mode. The sample is the same as in figures 5 and 6, single Rhodamine 6G molecules in polyvinyl alcohol on a metal-coated glass cover slide.



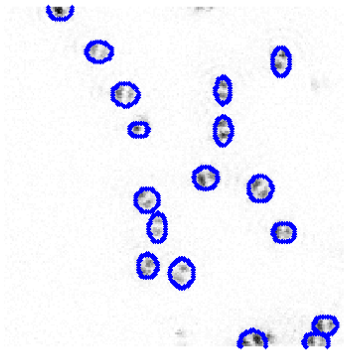
(a) Photons in pixels with >25 photons.



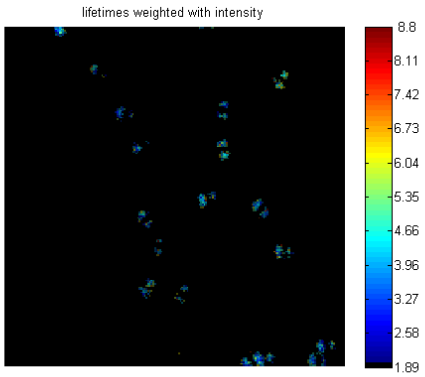
(b) Raw intensity image.



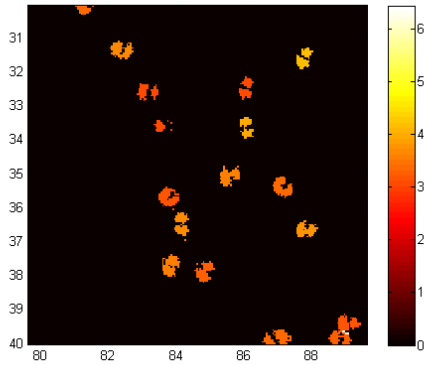
(c) Lifetimes of single pixels.



(d) Regions of interest (ROIs) selected by user.



(e) Lifetimes, brightness encodes intensity.



(f) Lifetimes of ROIs.

Figure 9 – Atto 655 molecules on a glass cover slide, surrounded by air. **Left column:** Evaluated in the pixel-by-pixel mode. **Right column:** Evaluated in the single-molecule mode using regions of interest (ROIs).

6 Extra tool: FinalAnalysisROI

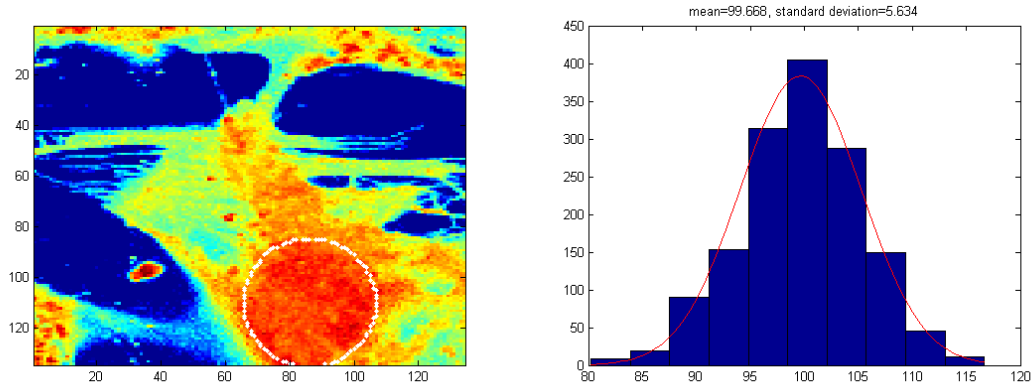


Figure 10 – Height results from figure 7 evaluated usign the tool `FinalAnalysisROI`.
Left: A pseudocolor image of the height results. The white diamonds mark the edge of the ROI.
Right: The height values of the pixels inside the ROI have been histogrammed, their mean and standard deviation are displayed in the title of the figure. The red curve is a Gaussian curve with the same mean and standard deviation.

When evaluating pixel-by-pixel data, one is often concerned with samples where areas of several pixels are at the same height. For a systematic analysis, one needs to know the statistics of the single-pixel results. This is the purpose of the function `FinalAnalysisROI`, which comes with the MIET-GUI software package. When running `FinalAnalysisROI(A)` in the MATLAB command window with an arbitrary array `A`, the array is plotted using `imagesc` and the user can draw as many elliptical regions of interest as needed. The pixels from all ROIs are then combined in a single histogram, the mean and standard deviation are calculated and a Gaussian curve with the same mean and standard deviation is displayed together with the histogram. The latter can be used to estimate if the fluctuations in the values are simply caused by noise or if there is some physical/biological reason why the height of the fluorescent molecules differs from pixel to pixel. As an optional second parameter, the number of bins for the histogram can be given (default is ten). Pixels which are `NaN` are ignored in all steps of the evaluation. Figure 10 shows the result when the height results from example three of the previous section are evaluated using `FinalAnalysisROI`.
In order to do this evaluation, one needs the quantity to be evaluated as a variable in the workspace. The easiest way of doing this is by loading the `.mat`-file containing the variable. In the single-pixel mode, this is the file ending on `PS.mat` for the intensity (`tag`) and lifetime (`life_imm`) and the file ending on `height.mat` for the height. Another possibility exists only while the GUI is still open, in this case, the intensity, lifetime, and height images can be accessed as the properties `Intenstiy`, `Lifetime` and `Height` of the GUI. For example, if the GUI has been started by typing `g=MIET_gui`, all properties are stored in `g`[5] and one can evaluate the height data via `FinalAnalysisRoi(g.Height)`.

[5] If one symply types `MIET_gui` to start the program, all properties are stored in `ans`.

7 Extra tool: PlotBrightAreas

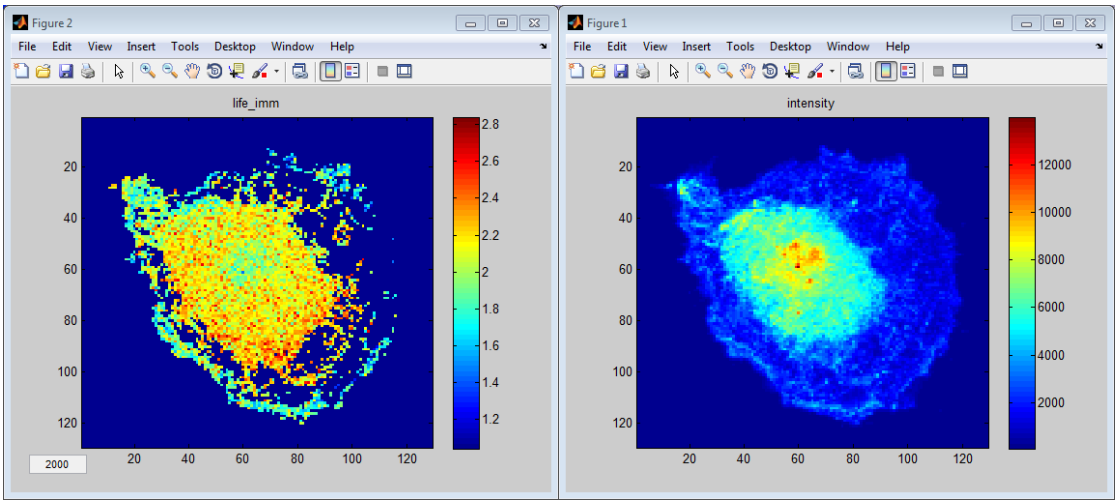


Figure 11 – Snapshots from the program `PlotBrightAreas`. **Left:** The lifetime image of a stained cell is shown, pixels with an intensity of less than 2000 photons are marked as `NaN` and are thus coloured deep blue. **Right:** Intensity image of the same cell, giving the user an idea which values are sensible for the threshold.

The calculated lifetimes and thus heights are generally less accurate in pixels with very few photons. The fitting algorithm automatically only considers pixels with at least 500 photons but there may be cases where one is only interested in pixels with even higher photon counts. The function `PlotBrightAreas`, which comes with the MIET-GUI software package, allows the user to interactively set an intensity threshold for displaying lifetime or height information.
The program is called as `PlotBrightAreas(A,B,flag)`, where `A` is an array containing the "interesting quantity" (i.e. the lifetime or height information) and `B` is the array containing the intensity image. Obviously, both arrays have to have exactly the same size. If `flag` is set to a nonzero value, the intensity image is plotted as well as the image of the "interesting quantity", if `flag` is not given, empty or zero, only the latter image is plotted.
In the image of the "interesting quantity", there is an editable text field at the bottom left corner. The user can write any number into this field and then press enter or click the mouse anywhere outside the text field to set a new threshold. The image is immediately updated such that all pixels with an intensity smaller than the threshold are shown as "not a number".

8 List of main functions

1. **Axial_error.m:** Calculates the height uncertainty using the lifetime estimation error.

2. **Axial_pos.m:** This program calculates the axial location of single molecules, given their polar angle, quantum yield and lifetime.

3. **BG_val.m:** Calculates the mean background signal of the TCSPC curve.

4. **Calc_mIRF.m:** Calculates parametrized IRF based on the TCSPC curve.

5. **Collect_tcspc.m:** Constructs TCSPC decay curves of each individual single molecule from their corresponding identified pixels.

6. **CombineImages.m:** Visualize multiple images in a figure panel.

7. **Convol.m:** Performs a convolution of the instrumental response function irf with the decay function.

8. **Core_ScanRead.m:** Reads the *.ht3* file of an image scan and assigns the photons to the corresponding lines and pixels.

9. **DeadTimeCorrection.m:** Program for recovering an unbiased decay-curve from a measured TCSPC curve.

10. **DetectTimeGates.m:** Divides the TCSPC histogramm into time-windows of the same length in order to separate the data from different excitation pulses.

11. **DipoleL.m:** calculates the electric field amplitudes of dipole radiation along emission angle theta of a dipole at distance z from an interface within a layer. See additional documentation on LifetimeL

12. **DistFluoFit.m:** Performs a TCSPC fit using a set of distributed decay model curves.

13. **DistTailFit.m:** Performs a exponential fit of the TCSPC tail part using a set of distributed decay exponential decay curves.

14. **ExpFun.m:** The error function for a theoretical exponential decay curve against the original decay data.

15. **fastSM_FLIM.m:** The main function that processes raw *.ht3* or *.ptu* files, performs pattern matching of single molecule intensity scan images, collects photons and constructs TCSPC decays of individual molecules and determines the fluorescence lifetimes of each molecule.

16. **FinalAnalysisROI.m:** [val_mean,val_stdDev,field_val] = FinalAnalysisROI(A,nBin,B) accepts an array A of real values, plots them using imagesc and lets the user draw an arbitrary number of elliptical regions of interest. All pixels contained in these ROIs are collected in a single histogram and displayed together with a Gaussian curve with the same mean and standard deviation. The latter two parameters are displayed above the histogram and are returned by the function, the values of the pixels are returned as field_val. The number of bins used for the histogram can be given as the optional parameter nBin (default is nBin=10). Pixels with the value "NaN" are ignored in the analysis.Furthermore, it is possible to give a second array B of the same size as A which contains intensity information. This array is plotted using imagesc and the user is asked to choose a threshold: pixels whose values in B are smaller than the threshold will be ignored in the analysis of A.

17. **FindPatternRapid.m:** Takes the intensity image and the computed model patterns to identify single molecules.

18. **FLIM_Analysis.m:** This program analyses FLIM image and calculates the lifetime image by fitting lifetime curves for each pixel which has > 30 photons. name is the *.ht3* file which is to be analysed. IRF is the *.ht3* file which contains the Instrument Response. If not given, the program calculates an IRF using parametric equations

19. **FreeSpaceLifetime.m:** FreeSpaceLifetime(name) finds the free space lifetime of a dye by evaluating a sample consisting of this dye in the same medium that will be later used in the MIET experiment.

20. **FresnelL.m:** [rp,rs,tp,ts] = Fresnel($w1, n1, n2$) calculates Fresnel reflection (r) and transmission (t) coefficients for p-wave and s-wave, either for crossing one interface or the effective coefficients when crossing a stack of media. Since these coefficients depend on the angle of incidence of the wave onto the interface, the component $w1$ of the wave vector perpendicular to the interface in the first medium (e.g. the medium where the emitter is situated) is also needed. The component q parallel to the interface is the same in all media. We assume that the length scale is chosen in such a way that the wave vector in vacuum,$k_0 = omega/c$, has the length 1. Then, $k_1 = n_1, k_2 = n_2, n_1^1 = q^2 + w_1^2$ and so on.

21. **Gauss2D.m:** Fits a 2D Gaussian to an intensity image.

22. **GaussExc.m:** Calculates the electric field distribution of a focused laser beam.

23. **HashWaveguideMode.m:** Function determines the in-plane wave vector components for the waveguide modes.

24. **Harp_tcspc.m:** This program gives the TCSPC Histogram for *.ht3* files or *.ptu* files.

25. **HT3_Read.m:** This function reads single-photon data from a *.ht3* file.

26. **LifetimeLSimpsExp.m:** calculates the lifetime and quenching of an oscillating dipole at a distance z from an interface within a layer using Simpson's rule with exponential integration points for the integration of evanescent waves.

27. **lsfit.m:** returns the Least-Squares deviation between the data and the computed values.

28. **makenima.m:** "MAKE Nice IMAges" – creates one figure with two graphs in it: the top one is a 3D view of a height profile (data in "height") with its corresponding shadow (data in "shadow"); the bottom one is a side view of the 3D height profile.

29. **mHist.m, mHist2.m, mHist3.m:** Function for computing 1D, 2D, and 3D histograms.

30. **MIET_Analysis.m:** This function initiates the analysis of the data file (*.ht3* format).

31. **MIET_Calc.m:** MIET_Calc calculates the calibration curves for different angles given the single molecule's quantum yield, free lifetime and the wavelength. The program needs the layer information as inputs. All lengths in nm.

32. **MIET_lifedist.m** Estimates the distribution of decay constants in a multi-exponential TCSPC decay curve.

33. **mim.m:** Custom image plotting function.

34. **pattern_detect.m:** pattern_detect initiates pattern matching program and collects the image of identified patterns. The identified image is filtered against the background in the image.

35. **PixelTau.m:** Corrects the recorded TCSPC histogram for deadtime artefacts and fits the fluorescence lifetime. This routine is used to calculate the deadtime corrected fluorescence lifetime of each pixel in a FLIM image.

36. **Process_Frame.m:** This program constitutes the heart of fast processing *.ptu* FLIM scan raw data files. It estimates the fluorescence lifetime of pixels by estimating the variance of the arrival times of photons.

37. **Process_scan.m:** The main routine that is called by MIET-GUI for processing raw *.ht3* and *.ptu* files and calculate FLIM images. It calls functions **Core_ScanRead.m** and **PTU_ScanRead.m** depending on the raw file format.

38. **PTU_ScanRead.m** Reads the *.ptu* file of an image scan and assigns the photons to the corresponding lines and pixels.

39. **PTU_Read.m** This function reads single-photon data from a *.ht3* file.

40. **radialpatterns_pixel.m:** Computes patterns (radial, azumithal or 2D gaussian) for the given pixel size and the excitation wavelength and then identifies molecules by calling **FindPatternRapid.m**.

41. **Simplex.m:** [x, dx, steps] = Simplex('F', X0, XMIN, XMAX, TOL, STEPS, VARARGIN) attempts to return a vector x and its error dx, so that x minimzes the function F(x) near the starting vector X0 under the conditions that xmin <= x <= xmax. TOL is the relative termination tolerance dF/F; (default = $10e-10$) STEPS is the maximum number of steps; (default = 200*number of parameters). The returned value of STEPS is the actual number of performed steps. Simplex allows for up to 10 additional arguments for the function F. Simplex uses a Nelder-Mead simplex search method.

9 Example code for performing tail fitting of a raw data file

```
% Reading the TCSPC histogram

 [bin,tcspcdata,head] = Harp_tcspc(name); % name represents the raw data file (.ht3 or .ptu)
 % plotting the TCSPC curve

 resolution = head.Resolution; % the resolution of each time channel
 tau        = (1:numel(bin))*resolution; % tcspc time bins
 figure
 title('TCSPC decay')
 semilogy(tau,tcspcdata);
 xlabel('\tau (ns)')
 ylabel('Counts')

 % --- TCSPC fitting ---

 % What we perform below is a tail fitting.
 [~, pos] = max(tcspcdata); % time bin with maximum counts
 cutoff   = 1; % 1 ns cutoff
 shift = round(cutoff./resolution); % time bin from which we start fitting
 tcspc = sum(tcspcdata(shift+max(pos):end,:),2); %
 t    = [0:numel(tcspc)-1]*resolution;
 p = [2]; % initial guesses in ns. One can give multiple values for multiexponential fitting.

 figure
 p = Simplex('ExpFun',p,[],[],[],[],t,tcspc,1,[],1); % Fitting exponential functions

 disp(['The fitted values are  ' num2str(p.') ' ns'])
 [~, c, ~, z] = ExpFun(p, t,tcspc,1);
 mean_tau = sum(c(2:end).*p)/sum(c(2:end));
 clf
 plot(t,tcspc,'o',t,z,'r','LineWidth',2)
 axis tight
 text('Position',[numel(tcspc)*resolution*0.7,z(1)*0.9],'String',['\tau = ',mnum2str(mean_tau,[],2),' ns'],'FontSize',16)
 xlabel('\tau (ns)','FontSize',16)
 ylabel('Counts','FontSize',16)
 set(gca,'FontSize',14)

 % Average arrival time of all photons

 counts_end = mean(tcspc(end-11:end-1)); % Offset
 av_arriv_time = sum(tcspc.*t.'-counts_end*mean(t))./(sum(tcspc)-counts_end*mean(t)); % Average photon arrival time
```